

PA2 实验报告

231180166 杜明阳

实验进度

PA2实验任务全部完成，在inst.c完成了译码 & 执行操作；实现了字符串处理和输出；完成了调试基础设施trace部分的内容；输入输出设备的软硬件代码全部完成，ioe完全实现，拓展内容声卡部分也已经完成。

必答题

1. 程序是个状态机：理解YEMU的执行过程

在计算机系统中，**状态 = 所有寄存器 + 内存 + PC（程序计数器）**

对于 YEMU 来说，其完整状态可以表示为一个元组：

State = (pc, R[0], R[1], R[2], R[3], M[0..15])

而整个程序的执行过程，就是从初始状态开始，每条指令改变一次状态，直到终止。

要执行的程序（计算 $z = x + y$ ）

内存内容（M[0] ~ M[7]）如下：

地址	二进制	十进制	指令 / 数据
0	11100110	230	load 6
1	00000100	4	mov r1, r0
2	11100101	229	load 5
3	00010001	17	add r0, r1
4	11110111	247	store 7
5	00010000	16	x = 16
6	00100001	33	y = 33
7	00000000	0	z = ? (结果)

目标是将 x=16 和 y=33 相加，结果存入 z=M[7]

NEMU中的执行流程：

1. 主循环调用 `exec_once()`
NEMU的主循环中调用 `cpu_exec()->execute()`，后者循环调用 `exec_once()`，表示“执行一条指令”。
2. 取指：`inst = inst_fetch(&cpu.pc, 4)`
 - 调用 `inst_fetch()` 函数，从 `cpu.pc` 指向的地址读取4字节指令。
 - 通过 `swaddr_read()` 访问内存（可能是DRAM或IO设备）。
 - 返回指令编码，如 `0x003100B3`。
3. 译码：通过宏或函数解析指令
 - NEMU使用宏或函数（如 `decode_exec()`）将指令拆解为：
 - `funct7=0x00, rs2=x3, rs1=x2, funct3=0x00, rd=x1, opcode=0x33`
 - 根据这些字段匹配到 `add` 指令以及相关操作数。
4. 更新PC：`cpu.pc += 4`
 - 指令长度为4字节，PC加4，指向下一条指令。
5. 返回并继续循环
 - `exec_once()` 结束，回到 `cpu_exec()`，继续执行下一条指令。

3. 程序如何运行：理解打字小游戏如何运行

打字小游戏的运行依赖于**事件驱动机制**和**定时器刷新**。这个打字小游戏是一个运行在 AM（抽象机）平台上的简单字符击打游戏，其核心机制如下：

- 游戏目标**：在下落的字母触底前按下对应键将其“击中”。
- 基本流程**：
 - 字符从屏幕顶部随机位置生成并向下掉落；
 - 玩家按下对应字母键可击中字符，使其变绿并向上消失；
 - 若字符触底未被击中，则计为“miss”并短暂变红；

- 按错键则计为“wrong”。
- **技术实现：**
 - 使用 AM 提供的定时器驱动每秒 30 帧更新逻辑；
 - 键盘事件实时处理输入；
 - 显卡接口绘制字符和背景，支持颜色变化动画；
 - 字体预渲染为纹理数组，提升绘制效率。
- **设计特点：**
 - 采用“状态机”管理字符生命周期；
 - 分离逻辑更新与画面渲染；
 - 跨平台运行，依赖抽象设备接口。

核心流程如下：

1. 初始化：

- 调用 `__am_gpu_config()` 获取屏幕分辨率。
- 调用 `__am_audio_config()` 初始化音频（如果需要音效）。
- 初始化游戏状态：当前输入字符串、目标字符串、计时器、得分等。

2. 主循环：

- 循环调用 `__am_timer_uptime()` 获取当前时间。
- 调用 `__am_input_keybrd()` 检查是否有键盘输入。
 - 如果有按键，将其加入输入缓冲区，并与目标字符串比对。
 - 如果匹配，得分增加，生成新单词。
- 调用 `__am_gpu_draw_rect()` 或 `__am_gpu_draw_chars()` 在屏幕上绘制当前状态（单词、输入、得分）。
- 调用 `__am_gpu_update()` 刷新屏幕。

3. 事件处理：

- 每次循环都检查输入事件（键盘、鼠标）。
- 定时器用于控制单词下落速度或游戏节奏。

4. 结束条件：

- 达到目标得分或时间耗尽，调用 `__am_exit()` 退出。

4. 编译与链接：static inline inst_fetch() 的作用

在 `nemu/include/cpu/ifetch.h` 中：

```
static inline uint32_t inst_fetch(vaddr_t *pc, int n) { ... }
```

- **static**：使函数具有内部链接（internal linkage），每个翻译单元（.c）包含该头时都会得到自己私有的一份，不会导致链接冲突。
- **inline**：允许编译器内联以减少函数调用开销

我们分别尝试：

(1) 去掉 static

- **结果**：多个 .c 文件包含此头文件时，每个 .o 文件都会生成一个 `inst_fetch` 的**全局符号**。
- **链接时错误**：出现 `multiple definition of 'inst_fetch'`。
- **原因**：没有 `static`，函数成为外部链接（external linkage），链接器发现多个同名函数，无法决定使用哪一个。
- 理论上是这样，实际并没有看到报错

(2) 去掉 inline

- **结果**：函数不再是内联建议，编译器可能生成函数体。
- **但因为有 static**，每个 .o 文件都有一个**私有副本**，不会冲突。
- **不会报错**，但可能导致代码膨胀（每个 .o 都有一份函数体）。

(3) 去掉 static 和 inline

- **结果**：函数成为外部链接且非内联。
- **链接错误**：`multiple definition of 'inst_fetch'`。
- **原因**：多个 .o 文件都定义了同名全局函数。

```
usr/bin/ld: /home/tony/codingproj/ics2025/nemu/build/obj-riscv32-nemu-interpreter/src/engine/interpreter/hostcall.o: in function
/home/tony/codingproj/ics2025/nemu/include/cpu/ifetch.h:20: multiple definition of `inst_fetch'; /home/tony/codingproj/ics2025/
collect2: error: ld returned 1 exit status
```

5. 在 common.h 中添加 volatile static int dummy; 重新编译后，NEMU 中有多少个 dummy 实体？

- **答案**：每个包含 `common.h` 的 .c 文件都有一个 `dummy` 的副本。

- 因为 static 变量具有**内部链接 (internal linkage)**，每个编译单元独立拥有一个实例。
-

```
tony@tonystar-linux:~/codingproj/ics2025/nemu$ find build/obj-riscv32-nemu-interpreter -name '*.o' -print0 | xargs -0 nm -a | g
35
```

6. 再在 debug.h 中添加 volatile static int dummy;

此时：

- common.h 和 debug.h 各自定义了一个 static int dummy;
- 包含 common.h 的 .c 文件：有一个 dummy
- 包含 debug.h 的 .c 文件：有另一个 dummy
- 如果一个文件同时包含两者，则有两个独立的 dummy（名字相同但作用域不同）

实体数目

- 应该比上一题更多，因为 debug.h 被一些 common.h 未覆盖的 .c 文件包含。例如：cpu-exec.c 可能只包含 debug.h，不包含 common.h，它也会生成一个 dummy。
- 但实际上都是35，可能并没有上述未覆盖的情况

结论

- static 变量在每个编译单元中独立存在。
- 不同头文件中的 static 变量互不影响，即使名字相同。

7. 添加初始化：volatile static int dummy = 0; 重新编译，会发生什么？

```
In file included from /home/tony/codingproj/ics2025/nemu/include/common.h:48,
                  from src/nemu-main.c:16:
/home/tony/codingproj/ics2025/nemu/include/debug.h:23:21: error: redefinition of 'dummy'
   23 | volatile static int dummy=0;
      |                   ^~~~~
In file included from src/nemu-main.c:16:
/home/tony/codingproj/ics2025/nemu/include/common.h:37:21: note: previous definition of 'dummy' with type 'int'
   37 | volatile static int dummy=0;
      |                   ^~~~~
make: *** [/home/tony/codingproj/ics2025/nemu/scripts/build.mk:34: /home/tony/codingproj/ics2025/nemu/build/obj-riscv32-nemu-in
```

- **链接错误**：redefinition of 'dummy'
- 编译器在包含两个头的同一翻译单元报重复定义原因：

在同一翻译单元中，多个无初始值的 static int dummy;（或仅是声明/暂定定义）可以合并为一个 tentative definition；但一旦带有初始化（= 0），它们就是真正的定义。

如果某个 .c 同时包含了 common.h 和 debug.h（两处都含有 static int dummy = 0;），就会在同一 TU 中出现两个对同名符号的已定义（initialized）定义，导致编译器报重定义错误。

但为什么之前没问题？

- 之前未初始化时，dummy 是 **common symbol**（未初始化的全局/静态变量），链接器允许跨文件合并。
- 初始化后，它成为 **已定义的符号**，且 static 使其为局部符号，每个 .o 文件都有一个定义，初始化后就变成明确定义，冲突暴露。

8. Makefile：make ARCH=\$ISA-nemu 的过程

1. Makefile 的组织方式
 - 顶层 Makefile 使用变量（如 ARCH、CC、CFLAGS、INCLUDES 等）并通过 include 把针对不同 ARCH 的子规则/变量引入（例如 am/arch/\$(ARCH)/mk/ 下的 Makefile）。
 - 利用变量和模板，生成针对交叉编译器的命令（例如 riscv32-nemu 会设置交叉编译器、特定 CFLAGS、链接脚本等）。
 - Makefile 也会覆盖/重定义隐式规则（如 .c -> .o）来使用交叉编译器和特定选项。
2. 编译与链接过程（最终生成 build/hello-\$(ISA)-nemu.elf）
 - 预处理与编译：对每个 .c 文件运行 \$(CC) -c ... 生成 .o（使用交叉编译器和指定 include 路径）。
 - 汇编（若有 .S 文件）也生成 .o。
 - 链接：用交叉链接器/ld（或 gcc driver）和链接脚本（linker.ld）把所有 .o、静态库（如 libam.a、klib.a）链接成目标 ELF，设置入口点和内存布局。
 - 生成最终镜像（可能还会用 objcopy 生成二进制镜像）。